

Assignment 2: Test Automation Implementation

Deadline: Week 4

Objective

This assignment focuses on applying automated testing to the system and QA strategy you established in **Assignment 1**.

- Implement automated tests for high-risk components.
- Introduce quality gates to ensure test reliability.
- Collect metrics and reports to feed directly into your research paper/project.

Assignment Tasks

1. Automated Test Implementation

Objective:

Implement automated tests for the **critical/high-risk modules** identified in Assignment 1. Ensure tests are **maintainable, repeatable, and well-documented**, with proper version control tracking.

Step 1: Identify Test Scope

List all high-risk modules and functions to be automated. Fill in the table below:

Module/Feature	High-Risk Function	Test (High/Medium/Low)	Priority	Notes/Expected Outcome
Authentication	User login validation	High		Must handle valid/invalid credentials, session mgmt
Authentication	Token management	High		JWT token generation, refresh, expiry
Forms	Dynamic form validation	High		Required field validation, error messages
Forms	Form submission	High		Submit on valid state, prevent on invalid

API	POST /auth/login endpoint	High	Status 200/401, JSON response
API	Response contract validation	High	Token presence, user object, expiration
Dashboard	Navigation after login	High	Redirect to /gan-models, protected routes
Dashboard	Logout functionality	High	Token cleanup, redirect to login
Performance	Page load time	Medium	LCP < 3000ms
Performance	API response time	Medium	TTFB < 2000ms

Step 2: Define Test Cases

Document test cases for each module/function. Include input, expected result, and scenario type.

Test Case ID	Module/Feature	Description	Input Data	Expected Result	Scenario Type	Notes
TC01	Authentication	Valid login	demo@forensics.gov / demo123	Dashboard loads	Positive	Critical path
TC02	Authentication	Invalid password	demo@forensics.gov / wrong	Error message	Negative	Security check
TC03	Authentication	Non-existent user	fake@test.com / pass	Generic error (no enumeration)	Negative	Security / Privacy
TC04	Authentication	Empty email	(blank) / demo123	Validation error	Negative	Form rule

TC05	Authentication	Empty password	demo@forensics.gov / (blank)	Validation error	Negative	Form rule
TC06	Authentication	Invalid email format	invalid-email / demo123	Validation error	Negative	Format check
TC07	Authentication	Short password	demo@forensics.gov / short	Validation error	Negative	Length rule
TC08	Registration	New user registration	new@test.com / Pass123 / Name	User created, 201	Positive	Extension test
TC09	Registration	Duplicate email	demo@forensics.gov / Pass	409 Conflict	Negative	Uniqueness check
TC10	Registration	Missing fields	(blank) / (blank)	400 Bad Request	Negative	Validation
TC11	Forms	Required field validation	Submit empty form	Error shown	Negative	Form state
TC12	Forms	Valid form submission	Name/Email populated	Submit enabled	Positive	Form state
TC13	Forms	Invalid email warning	invalid-email in email field	Validation error	Negative	Format
TC14	Forms	Multi-select handling	Select checkbox items	Items checked	Positive	Select
TC15	API	Login request payload	Email/password POST	200/201 status	Positive	Contract test
TC16	API	Auth token response	POST /auth/login	Token in response	Positive	Contract test
TC17	Dashboard	Dashboard navigation	Login success	Redirect to /gan-models	Positive	Routing
TC18	Dashboard	Logout functionality	Click logout	Redirect to /login	Positive	Cleanup
TC19	Performance	Page load time	Navigate to /login	Load time < 3000ms	Positive	SLA check
TC20	Performance	API response time	GET /api/metrics	Response time < 2000ms	Positive	SLA check

Step 3: Track Script Implementation

Log details about each automated script written, including framework and status.

Script ID	Module/Feature	Automation Framework	Script Name/Location	Status	Comments
S01	Authentication	Playwright/TypeScript	tests/auth/login.spec.ts	Complete	TC01-TC07, 9 test cases, POM pattern
S02	Registration	Playwright/TypeScript	tests/auth/register.spec.ts	Complete	TC08-TC10, 3 test cases, API integration
S03	Forms	Playwright/TypeScript	tests/forms/form-validation.spec.ts	Complete	TC11-TC14, 4 test cases, component interaction
S04	API	Playwright/TypeScript	tests/api/auth-api.spec.ts	Complete	TC15-TC16 + Extended, 6 test cases, contract tests
S05	Dashboard	Playwright/TypeScript	tests/dashboard/dashboard.spec.ts	Complete	TC17-TC18, 2 test cases, navigation flows
S06	Performance	Playwright/TypeScript	tests/performance/performance.spec.ts	Complete	TC19-TC20, 2 test cases, metric tracking

Step 4: Version Control Tracking

Record commits and progress in Git/GitLab. This helps track development history.

Commit Hash	ID / Date	Module/Feature	Description of Changes	Author
28dd7dd	2026-04-04	Assignment 2	The source code for assignment 2 were added.	TheKhegaPlay
6e61810	2026-04-04	Assignment 2	The source code for assignment 2 were added.	TheKhegaPlay

```
PS B:\GitHubRepos\GAN-main> git log --oneline --pretty=format:"%h|%ad|%s|%an" --date=short
28dd7dd|2026-04-04|Assignment 2|TheKhegaPlay
6e61810|2026-04-04|Assignment 2|TheKhegaPlay
PS B:\GitHubRepos\GAN-main>
```

Step 5: Evidence for Research Paper

Track screenshots, logs, and notes for reproducibility and reporting metrics:

Evidence ID	Module/Feature	Type	Description	File Location/Link
E01	Authentication	Screenshot	Successful login dashboard	Note: No successful login screenshot available (auth tests passed but no dashboard screenshot captured)
E02	Authentication	Log	Login test execution log with timestamps	playwright-report/index.html (detailed test execution logs)
E03	API	Code Snippet	TC15 login endpoint test	tests/api/auth-api.spec.ts (line 53)
E04	API	Log	API test execution results	playwright-report/index.html (API test results and failures)
E05	Forms	Code Snippet	TC11-TC14 form validation tests	tests/forms/form-validation.spec.ts
E06	Forms	Screenshot	Form validation errors displayed	test-results/forms-form-validation-Form-92679-i-select-and-check-UI-items-chromium/test-failed-1.png

E07	Dashboard	Code Snippet	TC17-TC18 navigation tests	tests/dashboard/dashboard.spec.ts
E08	Dashboard	Log	Dashboard test execution log	playwright-report/index.html (dashboard test logs)
E09	Performance	Code Snippet	TC19-TC20 performance metrics	tests/performance/performance.spec.ts
E10	Performance	Log	Performance test results	playwright-report/index.html (performance test results)
E11	Test Data	Code Snippet	Centralized test data configuration	tests/utis/test-data.ts
E12	Test Utils	Code Snippet	Page Object Model LoginPage	tests/pages/LoginPage.ts
E13	Configuration	Code Snippet	Playwright configuration	playwright.config.ts
E14	Test Results	Screenshot	Test execution summary	test-results/forms-form-validation-Form-cla85--form-when-fields-are-valid-chromium/test-failed-1.png
E15	Test Results	Log	Detailed test execution log	playwright-report/index.html
E16	Test Results	CSV	Coverage metrics export	coverage/index.html (coverage report)
E17	CI/CD	Code Snippet	GitHub Actions workflow	.github/workflows/e2e-tests.yml

Metrics:

- Test Scope: 10 high-risk modules
- Test Cases: 27 implemented
- Coverage: 70% (7/10 modules automated)
- Passing: 12/27 (44.4%)
- Failing: 15 (requiring fixes)

2. Quality Gate Definition & Integration

2. Quality Gate Definition & Integration

Objective:

Define and implement **quality gates** for your automated tests to ensure reliability, coverage, and performance. Integrate tests into a **CI/CD pipeline** with clear alerting and failure-handling procedures.

Step 1: Define Pass/Fail Criteria

Establish the thresholds and criteria that automated tests must meet to pass. Fill in the table below:

Quality Gate ID	Metric / Criterion	Threshold / Requirement	Importance	Observed Results	Notes
QG01	Test Pass Rate	$\geq 95\%$	High	44.4% (12/27)	Below threshold - failures require investigation
QG02	Code Coverage	$\geq 85\%$	High	70% (7/10 modules)	Below threshold
QG03	Critical Path (Auth)	100% for main flow	High	80% (8/9 passed)	Auth working, API failures impact dashboard
QG04	API Contract Validation	POSTs return JSON	High	17% (1/6 passed)	API endpoint 404 issue
QG05	Test Execution Time	≤ 10 min per module	High	~2.5 min	Good performance
QG06	Regression Test Success	100% for critical paths	High	44%	Multiple failures in API/Forms
QG07	Form Validation	100% per requirements	High	0% (0/4)	Form selectors not finding elements
QG08	Error Message Display	Correct error shown	Medium	100% (Auth tests)	Login errors display correctly
QG09	Performance - Page Load	LCP < 3s	Medium	0.924s	Exceeds SLA
QG10	Performance - API Response	TTFB < 2s	Medium	Timeout	Metrics endpoint not responding

Step 2: Integrate Tests into CI/CD Pipeline

Automate execution of tests in your chosen CI/CD system:

Pipeline Step	Description	Tool / Framework	Trigger (On Commit / Scheduled / PR)	Notes

Step 1	Checkout code	GitHub Actions	On commit to main	Get latest source
Step 2	Install dependencies	npm ci	Automatic	Clean install
Step 3	Build application	npm run build:ssr	Automatic	Compile browser + server
Step 4	Run automated tests	npx playwright test	Automatic	Execute all test suites
Step 5	Generate test reports	HTML/JSON reporters	Automatic	Create detailed reports
Step 6	Check quality gates	Custom script	Automatic	Verify pass rate, coverage
Step 7	Upload artifacts	GitHub Actions	Automatic	Save logs, screenshots
Step 9	Send notifications	Slack/Email	On failure	Alert team

Step 3: Document Alerting & Failure Handling Procedures

Define how your pipeline reacts to failures and how the team should respond:

Scenario / Event	Alert Type	Recipient / Channel	Action Required	Notes
Critical test failure	Email + Slack	QA Lead, Dev Team	Investigate failure, rerun test	Include logs/screenshots

API endpoint 404	Slack	Backend Team	Verify /api/auth/login route exists	High priority
Form selector timeout	GitHub Issue	QA Team	Update selectors, inspect DOM	Component visibility issue
Coverage below 85%	Email	Dev Team	Add test cases, increase coverage	Monitor trend
Test execution timeout	Pipeline Log	DevOps	Optimize slow tests	Multiple form tests timeout
Quality gate failure	GitHub Status	QA Team	Block merge until fixed	Prevents bad deploys
All tests pass	Slack	Team	Proceed with deployment	Success notification
Performance SLA missed	Slack	Performance Team	Analyze bottlenecks	API timeout issue

Step 4: CI/CD Pipeline Documentation

Students should **capture screenshots or diagrams** of their pipeline including:

- Steps of the pipeline.
- Test execution status.
- Notifications or alerts setup.

Deliverables:

1. **Quality Gate Report** – Include definitions, thresholds, and results for each quality gate.
2. **CI/CD Pipeline Screenshots / Diagram** – Show pipeline integration of automated tests.
3. **Alerting & Failure Handling Documentation** – Explain how failures are detected, reported, and resolved.

Best Practices

- Automate **critical gates first**, then expand to medium/low-risk metrics.
- Ensure pipelines are **reproducible and version-controlled**.
- Include **logging and reporting** at each step to track failures, coverage, and execution time.

- Update quality gates and CI/CD configuration based on test results and system changes

3. Metrics Collection

Objective:

Collect and analyze metrics from your automated tests to evaluate **effectiveness, coverage, and performance**. These metrics will feed directly into your research paper/project.

Step 1: Track Automation Coverage

Measure the percentage of **high-risk modules/functions** that have automated test scripts.

Module/Feature	High-Risk Function	Test Automated? (Yes/No)	Coverage %	Notes
Authentication	User login validation	Yes	100%	TC01-TC07 all implemented and running
Authentication	Token management	Yes	100%	JWT token tests included
Forms	Dynamic form validation	Yes	0%	TC11-TC14 fail due to selector timeout
Forms	Form submission	Yes	0%	Blocked by form visibility issues
API	POST /auth/login	Yes	0%	Returns 404, endpoint routing issue
API	Response contract	Yes	0%	Blocked by 404 response
Dashboard	Navigation after login	Yes	0%	Blocked by auth API failure
Dashboard	Logout functionality	Yes	0%	Blocked by auth API failure
Performance	Page load time (LCP)	Yes	100%	TC19 passing, 924ms

Performance	API response time	Yes	0%	TC20 metrics issue, timeout, endpoint
-------------	-------------------	-----	----	---------------------------------------

Formula:

Automation Coverage (%) = (Number of automated high-risk functions / Total high-risk functions) × 100

Coverage Formula: Automated Total = 7 automated / 10 total = **70% Coverage**

Step 2: Track Execution Time (TTE)

Record how long it takes to execute your automated tests. Include both **total execution time** and **per-module time**.

Module/Feature	Number of Test Cases	Execution Time per Test Case (sec)	Total Execution Time (sec)	Notes
Authentication	9	1400, 3300, 3600, 2300, 2300, 2400, 2500, 6100, 1000	25 sec	Includes extended + debug
Forms	4	16700, 16800, 16800, 16900	68 sec	Long timeout suggests DOM issues
API	6	50, 55, 50, 56, 50, 40	5 sec	Fast but all fail on assertion
Dashboard	2	1300, 26900	28 sec	Second test has long timeout
Registration	3	340, 344, 325	1 sec	Quick API tests
Performance	2	924, timeout	17 sec	One passes, one times out

Tip: Track execution time for both **CI/CD runs** and **local runs** to identify performance bottlenecks.

TTE Analysis:

- Fastest: API tests (50ms)
- Slowest: Form tests (16700ms) - timeout issues
- Average: 5.5 seconds per test

- Target: ≤ 10 minutes (600s) → Achieved: 2.5 minutes

Step 3: Track Defects Found vs Expected Risk

Compare actual defects discovered during automation with the **expected risk levels** identified in Assignment 1.

Module/Feature	High-Risk Level (High/Medium/Low)	Expected Defects	Defects Found	Pass/Fail	Notes
Authentication	High	2	0	Pass	All auth tests passing as expected
Forms	High	3	4	Fail	More failures than expected: selector and timeout issues
API	High	2	5	Fail	Endpoint 404 is root cause of 5 failures
Dashboard	High	2	2	Fail	Blocked by auth API failure
Registration	Medium	1	3	Fail	Depends on working API endpoint
Performance	Medium	1	1	Fail	Metrics API endpoint not responding

Defect Summary:

- Total Expected: 11 defects
- Total Found: 15 defects
- Defect Detection Rate: 136% (found more than expected)
- Root Causes: API routing (404), form visibility, endpoint timeouts

Step 4: Maintain Detailed Logs

For every test execution, keep a **log capturing**:

- Date/time of execution
- Test case ID / Script ID
- Result (Pass/Fail)
- Defects discovered
- Execution time per test
- Notes on failures or anomalies

Test Case ID	Module/Feature	Execution Date/Time	Result	Defects Found	Execution Time (ms)	Notes
TC01	Authentication	2026-04-03 10:00	Pass	0	1400	Valid login successful
TC02	Authentication	2026-04-03 10:01	Pass	0	3300	Invalid password error shown
TC03	Authentication	2026-04-03 10:02	Pass	0	3600	Non-existent user handled
TC04	Authentication	2026-04-03 10:03	Pass	0	2300	Empty email validation
TC05	Authentication	2026-04-03 10:04	Pass	0	2300	Empty password validation
TC06	Authentication	2026-04-03 10:05	Pass	0	2400	Invalid email format
TC07	Authentication	2026-04-03 10:06	Pass	0	2500	Short password validation
TC08	Registration	2026-04-03 10:07	Fail	1	340	API endpoint 404
TC09	Registration	2026-04-03 10:08	Fail	1	344	API endpoint 404
TC10	Registration	2026-04-03 10:09	Fail	1	325	API endpoint 404
TC11	Forms	2026-04-03 10:10	Fail	1	16700	Form selector timeout
TC12	Forms	2026-04-03 10:11	Fail	1	16800	Form input not found
TC13	Forms	2026-04-03 10:12	Fail	1	16800	Email validation timeout
TC14	Forms	2026-04-03 10:13	Fail	1	16900	Multi-select timeout
TC15	API	2026-04-03 10:14	Fail	1	50	API status 404 instead of 200

TC16	API	2026-04-03 10:15	Fail	1	55	No token in 404 response
TC17	Dashboard	2026-04-03 10:16	Fail	1	1300	Dashboard not loading
TC18	Dashboard	2026-04-03 10:17	Fail	1	26900	Logout timeout
TC19	Performance	2026-04-03 10:18	Pass	0	924	Page load 924ms < 3s
TC20	Performance	2026-04-03 10:19	Fail	1	15000+	Metrics API timeout

Step 5: Metrics Reporting

Metric	Template Target	Notes
Automation Coverage	$\geq 85\%$	Results depend on your app scope
Test Pass Rate	$\geq 95\%$	Based on correct selectors/data
Execution Time (TTE)	≤ 8 min	Your own test suite timing
Quality Gates	10/10	Report demonstrates all gates

- Compile **tables and charts** from Steps 1–4:
 - Bar chart: Automation coverage per module
 - Line chart: Test execution times per module
 - Table: Defects found vs expected risk

Deliverables:

1. **Metrics Report** – Include all tables, charts, and logs showing coverage, execution times, and defects.
2. **Evidence for Research Paper** – Screenshots of metrics dashboards or exported CSV/HTML reports from automation tools.

4. Documentation

Objective:

Update your **QA Test Strategy Document** to include all automation details, quality gates, CI/CD integration, and metrics. This will serve as a foundation for the **research paper's methodology and results sections**.

Step 1: Automation Approach & Tool Selection

Document your **automation strategy** and justify your choice of tools.

Section	Details to Fill In	Example / Prompt
---------	--------------------	------------------

Automation Approach	Describe your testing strategy (e.g., risk-based, regression-focused, end-to-end testing)	Risk-based approach automating high-risk modules first with emphasis on regression testing and critical path validation
Tool Selection	List the tools/frameworks used and reason for selection	Playwright + TypeScript for native TypeScript support, built-in API testing capabilities, CI/CD friendly, cross-browser support (Chromium, Firefox, WebKit)
Scope	Which modules/functions were automated	Selenium (legacy), Cypress (limited API testing), WebdriverIO (heavier, more complex setup)
Reusability	How scripts are modular and maintainable	10 high-risk modules: Authentication (login, token mgmt), Registration, Forms (validation, submission), API (/auth/login), Dashboard (navigation, logout), Performance (page load, API response)

Step 2: Quality Gate Definitions

Include the **pass/fail criteria** established for automated testing.

Quality Gate ID	Metric / Criterion	Threshold	Observed Results	Notes
QG01	Test Pass Rate	$\geq 95\%$	44.4% (12/27)	Failed - Below threshold; API endpoint 404 and form selector issues are root causes
QG02	Code Coverage	$\geq 85\%$	70% (7/10 modules)	Below threshold - 3 modules blocked by API routing issues
QG03	Critical Path (Auth)	100% for main flow	89% (8/9 passed)	Partial pass - Authentication working; API failures block downstream tests
QG04	API Contract Validation	POSTs return valid JSON	17% (1/6 passed)	Failed - POST /api/auth/login returns 404 instead of 200/401
QG05	Test Execution Time	≤ 10 min per module	2.5 min total	Passed - Excellent performance; form test timeouts don't exceed limit
QG06	Regression Test Success	100% for critical paths	44%	Failed - Multiple failures in API, Forms, Registration, Dashboard

QG07	Form Validation	100% per requirements	0% (0/4 passing)	Failed - Playwright cannot locate form inputs with formcontrolname selectors
QG08	Error Message Display	Correct error shown	100% (Auth tests)	Passed - All authentication error messages display correctly
QG09	Performance - Page Load	LCP < 3s	0.924s	Passed - Exceeds SLA requirement significantly
QG10	Performance - API Response	TTFB < 2s	Timeout	Failed - /api/metrics endpoint not responding or unreachable

Step 3: CI/CD Integration Overview

Document the CI/CD setup for automation.

Pipeline Step	Tool Framework	Trigger	Description	Status
Step 1: Checkout Code	GitHub Actions	On commit to main	Retrieve latest source code from repository	Configured
Step 2: Install Dependencies	npm ci	Automatic (after checkout)	Clean install of node_modules for reproducibility	Configured
Step 3: Build Application	npm run build:ssr	Automatic (after install)	Compile browser and server bundles using Angular SSR	Configured
Step 4: Run Automated Tests	npx playwright test	Automatic (after build)	Execute all 27 test cases across 6 test suites (chromium, firefox, webkit)	Configured
Step 5: Generate Test Reports	HTML/JSON reporters	Automatic (after tests)	Create detailed reports with screenshots, videos, and execution logs; publish to playwright-report/	Configured

Step 6: Check Quality Gates	Custom script	Automatic (after reports)	Verify pass rate ($\geq 95\%$), coverage ($\geq 85\%$), execution time ($\leq 10\text{min}$), regression success (100%)	3/10 Passed
Step 7: Upload Artifacts	GitHub Actions	Automatic (after quality check)	Archive test results, screenshots, videos, and logs to GitHub artifacts for debugging	Configured
Step 8: Send Notifications	Slack/Email	On failure only	Alert QA Lead and Dev Team of pipeline failures with links to reports and logs	Configured

Tip: Include workflow diagrams showing **pipeline steps**, **trigger points**, and **test execution flow**.

Step 4: Initial Results & Coverage Metrics

Include tables and charts summarizing **automation effectiveness**.

Module/Feature	Automated ?	Coverage %	Execution Time (sec)	Defects Found	Pass/Fail	Notes
Authentication	Yes	100%	25	0	Pass	TC01-TC07: 9 tests all passing, critical path functional
Performance (Page Load)	Yes	100%	17	0	Pass	TC19 passing with 924ms LCP, exceeds SLA

Registration	Yes	33%	1	3	Fail	TC08- TC10: API endpoint 404 blocks all tests
Forms	Yes	25%	68	4	Fail	TC11- TC14: Form selector timeouts (16700ms avg)
API	Yes	17%	5	5	Fail	TC15- TC16: POST /auth/logi n returns 404 instead of 200
Dashboard	Yes	0%	28	2	Fail	TC17- TC18: Blocked by auth API failure
Performance (API Response)	Yes	0%	N/A	1	Fail	TC20: Metrics endpoint timeout

Coverage Summary:

- **Total Modules:** 10 high-risk modules
- **Automated:** 7 modules (70% coverage)
- **Overall Execution Time:** ~150 seconds total (2.5 minutes) — Well within 10-minute limit
- **Total Defects Found:** 15 defects vs 11 expected (136% detection rate)
- **Pass Rate:** 12/27 tests (44.4%)

Step 5: Evidence for Reproducibility

Include **logs, screenshots, and code snippets** demonstrating how the automation can be rerun.

Evidence ID	Module/Feature	Type	Description	File Location/Link
E01	Authentication	Screenshot	Successful login flow screenshot	test-failed-1.png
E02	Authentication	Code Snippet	Login test implementation with POM pattern	login.spec.ts (lines 1-50)
E03	Authentication	Log	Authentication test execution log with timestamps	index.html (TC01-TC07 results)
E04	API	Code Snippet	API contract test for POST /auth/login	auth-api.spec.ts (lines 20-80)
E05	API	Log	API test failure details and error messages	index.html (TC15-TC16 results)
E06	API	Screenshot	HTTP 404 response from /api/auth/login	error-context.md
E07	Forms	Code Snippet	Form validation test implementation	form-validation.spec.ts (lines 1-60)
E08	Forms	Screenshot	Form selector timeout error screenshot	test-failed-1.png
E09	Forms	Log	Form test execution log with timeout details	index.html (TC11-TC14 results)
E10	Dashboard	Code Snippet	Dashboard navigation and logout tests	dashboard.spec.ts (lines 1-40)
E11	Dashboard	Log	Dashboard test execution results	index.html (TC17-TC18 results)
E12	Performance	Code Snippet	Performance metrics collection code	performance.spec.ts (lines 1-50)
E13	Performance	Log	Performance test execution with timing data	index.html (TC19-TC20 results)
E14	Configuration	Code Snippet	Playwright configuration with webServer and reporters	playwright.config.ts
E15	Test Data	Code Snippet	Centralized test data configuration	test-data.ts
E16	Page Object	Code Snippet	LoginPage POM implementation	LoginPage.ts
E17	CI/CD	Code Snippet	GitHub Actions workflow pipeline	e2e-tests.yml

Evidence Summary:

- **Total Evidence Items:** 17
- **Code Snippets:** 6 items (implementation reference)
- **Screenshots:** 2 items (visual failures)
- **Execution Logs:** 7 items (detailed results)
- **Configuration:** 2 items (setup reference)
- **Reproducibility Support:** All test code, config, and logs available for peer verification